

Resumo das apresentações – Felipe do Amparo Lauria

Grupo 2 (Renan e Breno) – Word Counter em Big Data: Esse grupo comparou contagem de palavras sequencial vs paralela num dataset gigante, quase 20 GB da Wikipedia em inglês, usando streaming em blocos pra não estourar a RAM. Para base pequena, com 1.000 registros, o sequencial ganhou, o overhead do paralelo não compensa nesses casos. Mas quando o dataset escala para 1 milhão de registros, o multiprocessing disparou na frente, reduzindo o tempo quase na metade. Ou seja, MapReduce caseiro só vale a pena quando o volume é grande.

Grupo 3 (Bruna e André) – MapReduce com a Wikipédia PT: Esse trabalho rodou WordCount num dump de 15 GB da Wikipédia em português, comparando Python sequencial contra PySpark rodando no Dataproc com clusters de 2, 4 e 8 workers. O sequencial demorou quase uma hora, enquanto o distribuído levou apenas 11 minutos, mas não foi ganho linear por causa da Lei de Amdahl. O grupo identificou gargalos, na utilização de RAM que impactou diretamente no uso da CPU, ficando mais tempo esperando dado chegar do que processando. No final a conclusão foi que distribuído só compensa em bases grandes e 4 workers foi o ponto mais equilibrado entre custo e desempenho.

Grupo 4 (Felipe Possari, Mario e Enrico) – Cryptography Cracker: Esse trabalho foi sobre quebrar hash MD5 via dicionário, comparando um script sequencial simples com uma versão distribuída usando FastAPI, Celery, Redis e PostgreSQL rodando em Docker. O sequencial processa a wordlist linha por linha e é fácil de medir, só que trava fácil quando a lista é grande. Já a versão com fila divide tudo em chunks de 50 mil linhas, roda em até 8 workers em paralelo, e tem um esquema de early abort, que assim que acha a senha em algum chunk já cancela o resto pra não ficar processando à toa. A coordenação entre os workers é feita via Redis, com uma flag que avisa quando já achou a senha. No geral o ganho de performance vem tanto de paralelizar quanto desse corte antecipado.

Grupo 5 (Raí e Luís) – Sistema de Arquivos P2P: Esse grupo construiu um sistema de compartilhamento de arquivo em P2P, focando em

concorrência e desempenho de rede. Tem um tracker central que não guarda os arquivos, só os metadados e a lista de quem tem cada arquivo (os seeders), e o cliente usa múltiplas threads e um heartbeat via UDP pra saber quem tá online. Nos testes, o modelo cliente servidor mostrou que quanto mais cliente entrava, mais o servidor virava gargalo, caindo o uso de banda. Já no modo P2P foi o contrário, mais seeders significava mais throughput, e no stress test com 4 seeders locais chegou perto de 4,7 Gbps. Deu pra ver bem que P2P escala muito melhor que o modelo centralizado.

Grupo 6 (Guilherme e Ana) – Sentimento em Tweets da Copa 2022: Esse trabalho foi focado fazer análise de sentimento em 5 mil tweets sintéticos sobre a Copa, rodando no Colab com 4 CPUs, usando TF-IDF e regressão logística. Só que o resultado surpreendeu, o distribuído com 4 workers ficou praticamente igual ao sequencial, até um pouquinho pior, porque a etapa de predição precisa mandar o modelo treinado pra todo mundo e isso gerou um aumento absurdo de tempo. Os workers ficaram boa parte do tempo parado esperando essa comunicação. A acurácia bateu 100% mas isso é porque o dataset era sintético, então não reflete a realidade. A conclusão é que com um volume pequeno desse jeito não vale a pena distribuir, só compensa em escala bem maior ou com modelo mais pesado tipo rede neural.

Grupo 7 (Alex e José) – Multiplicação de Matrizes Distribuída: Esse trabalho foi sobre multiplicação de matrizes, que tem custo $O(N^3)$ e cresce rápido, testando um esquema master worker com Python, NumPy, sockets TCP rodando em instância t3.micro da AWS. O resultado foi meio inesperado, o sequencial ganhou em absolutamente todos os testes, o pior cenário até foi justamente o de 8 workers. A Lei de Amdahl se fez presente, a parte que não dá pra paralelizar (serializar e mandar dado pela rede) limitou bastante o ganho, ficando bem longe do teórico. Com 8 workers a eficiência ficou bem baixa, os workers passavam mais tempo esperando rede do que calculando de fato. E ainda teve um problema de memória, com matriz 4096x4096 o 1 GB de RAM da instância não aguentou e o OOM Killer matou o processo.

Grupo 8 (Victor Cipolato) – Treinamento de ML Sequencial vs Distribuído:

Nesse trabalho foi feito o treino de Random Forest no dataset HIGGS, um dataset de grande porte (cerca de 11 milhões de registros), testando amostras de 1M, 3M, 5M e 11M. Comparou Scikit-learn rodando sequencial numa VM só do Colab com 10 GB de RAM contra Spark MLlib num cluster do Dataproc com 1 master e 2 workers. Diferente dos outros grupos, aqui o distribuído ganhou em todos os testes de volume grande, reduzindo a pressão de memória por máquina sem que o overhead de rede anulasse o ganho, e a acurácia ficou praticamente igual nos dois casos. Então pra esse tipo de dataset gigante, o distribuído realmente compensa o esforço a mais de configurar.

Grupo 9 (Vitor Almeida): Não achei o vídeo desse grupo.